

Exercise 1: Inventory Management System

1. Understanding the Problem

Why Data Structures and Algorithms are Essential

Data structures and algorithms help store, retrieve, update, and delete inventory data efficiently. They improve performance and scalability when handling large inventories.

Suitable Data Structures

- **ArrayList/List** – Easy to use but slower for searching.
- **HashMap/Dictionary** – Fast insertion, search, update, and deletion.
- **Linked List** – Efficient insertion and deletion.
- **TreeMap/Sorted Dictionary** – Maintains sorted data.

For this system, **HashMap (Dictionary)** is the most suitable choice.

2. Setup

Create a new project named **Inventory Management System**.

3. Implementation

Product Class Attributes

- ProductId
- ProductName
- Quantity
- Price

Data Structure Used

- **Dictionary<int, Product>**

Methods

- **Add Product** – Inserts a new product into inventory.
- **Update Product** – Modifies existing product details.
- **Delete Product** – Removes a product from inventory.

4. Analysis

Operation	Time Complexity
-----------	-----------------

Add Product	O(1)
-------------	------

Update Product	O(1)
----------------	------

Delete Product	O(1)
----------------	------

Optimization

- Use Dictionary instead of List for faster access.
- Use unique Product IDs as keys.
- Use database indexing for very large inventories.

Exercise 2: E-commerce Platform Search Function

1. Understanding Asymptotic Notation

Big O Notation

Big O notation measures the efficiency of an algorithm by describing how its execution time grows as the input size increases. It helps compare algorithms and choose the most efficient one.

Search Operation Cases

- **Best Case:** Element is found immediately. $\rightarrow O(1)$
- **Average Case:** Element is found somewhere in the middle.
- **Worst Case:** Element is found at the end or not found at all.

2. Setup

Product Class Attributes

- ProductId
- ProductName
- Category

3. Implementation

Linear Search

- Searches each product one by one in an array.
- Works on unsorted data.

Binary Search

- Searches by repeatedly dividing a sorted array into halves.
- Requires the array to be sorted.

Storage

- **Array** for Linear Search.
- **Sorted Array** for Binary Search.

4. Analysis

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

Exercise 3: Sorting Customer Orders

1. Understanding Sorting Algorithms

Bubble Sort

Compares adjacent elements and swaps them if they are in the wrong order. Repeats until the array is sorted.

Insertion Sort

Builds the sorted array one element at a time by inserting each element into its correct position.

Quick Sort

Selects a pivot element and partitions the array into smaller and larger elements, then recursively sorts the partitions.

Merge Sort

Divides the array into halves, sorts each half, and merges them back together.

2. Setup

Order Class Attributes

- OrderId
- CustomerName
- TotalPrice

3. Implementation

Bubble Sort

- Compare adjacent orders based on TotalPrice.
- Swap if the current order has a higher price than the next order.
- Continue until all orders are sorted.

Quick Sort

- Choose a pivot order.
- Place smaller-priced orders to the left and larger-priced orders to the right.
- Recursively sort both partitions.

4. Analysis

Algorithm Best Case Average Case Worst Case

Bubble Sort $O(n)$ $O(n^2)$ $O(n^2)$

Quick Sort $O(n \log n)$ $O(n \log n)$ $O(n^2)$

Why Quick Sort is Preferred

- Quick Sort is much faster for large datasets.
- It has an average time complexity of **$O(n \log n)$** compared to **$O(n^2)$** for Bubble Sort.
- It requires fewer comparisons and swaps.
- It is widely used in real-world applications due to its efficiency.

Exercise 4: Employee Management System

1. Understanding Array Representation

Array Representation in Memory

Arrays store elements in **contiguous memory locations**. Each element is accessed using its index, allowing fast retrieval of data.

Advantages of Arrays

- Fast access to elements using indexes.
- Simple to implement and use.
- Efficient memory utilization for fixed-size data.

2. Setup

Employee Class Attributes

- EmployeeId
- Name
- Position
- Salary

3. Implementation

Data Structure Used

- **Array of Employee Objects**

Operations

Add Employee

Insert a new employee record into the next available position in the array.

Search Employee

Search for an employee using Employee ID by traversing the array.

Traverse Employees

Visit and display all employee records stored in the array.

Delete Employee

Find the employee and remove the record by shifting subsequent elements to fill the gap.

4. Analysis

Operation	Time Complexity
Add Employee	$O(1)$
Search Employee	$O(n)$
Traverse Employees	$O(n)$
Delete Employee	$O(n)$

Where **n** is the number of employees.

Limitations of Arrays

- Fixed size; cannot grow dynamically.
- Insertion and deletion can be costly due to shifting elements.
- Searching is slower compared to hash-based structures.

When to Use Arrays

- When the number of records is known in advance.
- When fast index-based access is required.
- When memory efficiency and simplicity are important.

Exercise 5: Task Management System

1. Understanding Linked Lists

Singly Linked List

A Singly Linked List consists of nodes where each node contains data and a reference to the next node. Traversal is possible only in the forward direction.

Doubly Linked List

A Doubly Linked List consists of nodes where each node contains data and references to both the previous and next nodes. Traversal is possible in both directions.

2. Setup

Task Class Attributes

- TaskId
- TaskName
- Status

3. Implementation

Data Structure Used

- **Singly Linked List**

Operations

Add Task

Insert a new task node into the linked list.

Search Task

Traverse the linked list and find a task using its Task ID.

Traverse Tasks

Visit and display all tasks in the linked list.

Delete Task

Locate the task node and adjust links to remove it from the list.

4. Analysis

Operation	Time Complexity
------------------	------------------------

Add Task	$O(1)$
----------	--------

Search Task	$O(n)$
-------------	--------

Traverse Tasks	$O(n)$
----------------	--------

Delete Task	$O(n)$
-------------	--------

Where **n** is the number of tasks.

Advantages of Linked Lists over Arrays

- Dynamic size; no fixed capacity required.
- Efficient insertion and deletion without shifting elements.
- Better suited for applications where data changes frequently.
- Memory is allocated as needed.

Exercise 6: Library Management System

1. Understanding Search Algorithms

Linear Search

Linear Search checks each element one by one until the required book is found or the list ends. It works on both sorted and unsorted data.

Binary Search

Binary Search repeatedly divides a sorted list into two halves and searches in the appropriate half. It requires the data to be sorted.

2. Setup

Book Class Attributes

- BookId
- Title
- Author

3. Implementation

Linear Search

- Traverse the list of books sequentially.
- Compare each book title with the target title.
- Return the book if found.

Binary Search

- Store books in a sorted list based on title.
- Compare the target title with the middle element.
- Continue searching in the left or right half accordingly.
- Return the book if found.

4. Analysis

Algorithm	Best Case	Average Case	Worst Case
-----------	-----------	--------------	------------

Linear Search	$O(1)$	$O(n)$	$O(n)$
---------------	--------	--------	--------

Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$
---------------	--------	-------------	-------------

Where **n** is the number of books.

When to Use Each Algorithm

- **Linear Search**
 - Suitable for small datasets.
 - Works on unsorted data.
 - Easy to implement.
- **Binary Search**
 - Suitable for large datasets.
 - Requires data to be sorted.
 - Provides much faster searching.

Exercise 7: Financial Forecasting

1. Understanding Recursive Algorithms

Recursion

Recursion is a programming technique where a method calls itself to solve a problem by breaking it into smaller subproblems. It simplifies problems that have repetitive or self-similar structures.

2. Setup

Method

Create a recursive method to calculate future values based on:

- Present Value
- Growth Rate
- Number of Years

3. Implementation

Recursive Approach

The future value can be calculated using:

$$\text{Future Value} = \text{Present Value} \times (1 + \text{Growth Rate})$$

For multiple years, the recursive method repeatedly applies the growth rate until the required number of years is reached.

4. Analysis

Time Complexity

- Recursive solution: **$O(n)$**
- Where **n** is the number of years being forecasted.

Optimization

- Use **Memoization** to store previously computed results and avoid repeated calculations.
- Use an **Iterative Approach** for better memory efficiency.
- Use the direct formula:

$$\text{Future Value} = \text{Present Value} \times (1 + \text{Growth Rate})^n$$

to calculate the result in a single computation.